DOCUMENTACIÓN DEL PLAN DE DESARROLLO

Proyecto "IO"

Cronograma de implementación: 1 de octubre – 1 de diciembre

Propósito del documento
Describir de manera técnica y operativa cómo se ejecutará cada actividad contemplada en el cronograma, incluyendo objetivos, procedimiento, responsables, entregables, criterios de validación y dependencias.

Elemento	Definición
Frontend	Angular
Backend	.NET / API REST
Base de datos	SQL Server
Seguridad	Autenticación y autorización mediante JWT y roles
Tiempo real	Mecanismo de comunicación en tiempo real para el dashboard, según la arquitectura definida
Periodo	1 de octubre – 1 de diciembre
Equipo	Ángel, Emiliano, Miguel y equipo completo según la actividad

1. Información general

El cronograma establece nueve fases consecutivas para construir y poner en producción el sistema “IO”. La planificación se organiza por semanas y contempla una ejecución coordinada entre Angular, .NET y SQL Server. Aunque cada fase tiene una semana principal asignada, las actividades se consideran relacionadas: una fase debe dejar los componentes necesarios para que la siguiente pueda comenzar sin bloqueos.

1.1 Objetivo general

Desarrollar, integrar, validar y desplegar el sistema “IO” mediante una arquitectura web compuesta por un frontend en Angular, un backend en .NET y una base de datos SQL Server, incorporando autenticación segura, administración de equipos, búsqueda y edición, visualización en tiempo real, auditoría, pruebas y despliegue.

1.2 Principios de trabajo

- Trabajar por incrementos funcionales: cada semana debe producir un resultado comprobable.
- Mantener separación entre presentación, lógica de negocio, acceso a datos y seguridad.
- Definir contratos claros entre Angular y la API antes de integrar cada módulo.
- Validar cada módulo antes de considerarlo terminado.
- Registrar errores, decisiones técnicas y cambios relevantes para facilitar mantenimiento.
- Evitar que una actividad posterior dependa de trabajo no validado de una fase anterior.

1.3 Flujo general de desarrollo

1. Analizar el alcance de la actividad y confirmar sus entradas, salidas y dependencias.
2. Diseñar o ajustar la estructura necesaria en SQL Server, backend y frontend.
3. Implementar el componente correspondiente.
4. Integrar el componente con los módulos existentes.
5. Realizar pruebas funcionales y técnicas.
6. Documentar incidencias y corregir errores.
7. Validar el entregable con el responsable de la actividad.
8. Preparar la base para la siguiente fase.

1.4 Matriz general del cronograma

Fase	Semana	Periodo	Responsables
1. Arquitectura y BD SQL Server	S1	1–4 Oct	Ángel, Emiliano, Miguel
2. Autenticación JWT	S2	5–11 Oct	Ángel, Emiliano
3. Alta/Baja de Equipos	S3	12–18 Oct	Ángel, Miguel
4. Búsqueda y Edición	S4	19–25 Oct	Ángel, Emiliano
5. Dashboard (Real-time)	S5	26 Oct–1 Nov	Ángel, Emiliano
6. Roles y Logs (Auditoría)	S6	2–8 Nov	Emiliano, Miguel
7. Refinamiento DTUI	S7	9–15 Nov	Ángel
8. Pruebas Generales y QA	S8	16–22 Nov	Miguel (Líder), Equipo
9. Despliegue a Producción	S9	23 Nov–1 Dic	Equipo completo

1. Arquitectura y Base de Datos SQL Server

Semana programada: S1 — 1 al 4 de octubre | Responsables: Ángel, Emiliano y Miguel

Objetivo

Definir la arquitectura base del sistema y construir una base de datos SQL Server consistente, normalizada y preparada para soportar los módulos posteriores.

Descripción de la ejecución

Esta fase constituye el punto de partida técnico. Se establecerá cómo se comunicarán Angular, la API .NET y SQL Server, además de definir las entidades y relaciones necesarias para usuarios, equipos, estados, registros de auditoría y demás información requerida por el sistema.

Procedimiento de trabajo

9. Levantar las entidades funcionales que manejará el sistema y distinguir datos maestros, transaccionales y de auditoría.
10. Diseñar el modelo entidad-relación y revisar claves primarias, foráneas, restricciones, cardinalidades e índices.
11. Crear la base de datos y tablas en SQL Server utilizando scripts versionados.
12. Definir tipos de datos, valores por defecto, restricciones NOT NULL, UNIQUE y CHECK cuando correspondan.
13. Preparar índices para las consultas que serán utilizadas con mayor frecuencia, especialmente búsquedas por identificadores, nombres, estados y fechas.
14. Definir procedimientos, consultas o mecanismos de acceso a datos que realmente sean necesarios para la lógica del sistema.
15. Empezar a investigar como establecer la estructura del backend .NET y la comunicación con SQL Server.

Entregables

- Diagrama lógico/entidad-relación de la base de datos.
- Scripts SQL de creación y actualización de la BD.
- Base de datos inicial funcional en SQL Server.
- Documento breve de convenciones de nombres y relaciones.

Criterios de validación

- Las tablas y relaciones pueden crearse desde cero mediante scripts.
- No existen relaciones ambiguas o claves sin propósito.
- Las operaciones CRUD básicas sobre las entidades iniciales funcionan.
- El modelo soporta las necesidades de las fases 2 a 6.

Riesgos y consideraciones

- Cambios tardíos en el modelo pueden afectar endpoints y componentes Angular.
- Consultas sin índices adecuados pueden generar problemas de rendimiento.
- Guardar credenciales en el repositorio puede generar un riesgo de seguridad.

2. Autenticación JWT

Semana programada: S2 — 5 al 11 de octubre | Responsables: Ángel y Emiliano

Objetivo

Implementar un mecanismo de autenticación basado en JWT para identificar usuarios de forma segura y proteger los recursos de la API.

Descripción de la ejecución

La autenticación será la puerta de entrada a las funciones protegidas del sistema. El backend validará las credenciales y emitirá un token firmado. Angular conservará el estado de sesión de acuerdo con la estrategia de seguridad definida y enviará el token en las solicitudes protegidas.

Procedimiento de trabajo

16. Definir el modelo de usuario y los campos necesarios para autenticación.
17. Implementar el endpoint de inicio de sesión y validación de credenciales.
18. Configurar la generación de JWT con issuer, audience, expiración y firma mediante variables seguras.
19. Evitar devolver contraseñas o información sensible en las respuestas de la API.
20. Configurar middleware de autenticación y autorización en .NET.
21. Crear en Angular un servicio de autenticación para iniciar sesión, cerrar sesión y consultar el estado de sesión.
22. Implementar un interceptor HTTP para adjuntar el token a las peticiones protegidas.
23. Crear guards de navegación para impedir acceso a rutas que requieran autenticación.
24. Probar tokens válidos, expirados, ausentes o alterados.
25. Documentar el flujo de autenticación y los códigos HTTP esperados.

Entregables

- Endpoint de login funcional.
- Generación y validación de tokens JWT.
- Middleware de autenticación en .NET.
- Servicio, interceptor y guard de Angular.
- Documentación del flujo de autenticación.
- Casos de prueba de autenticación.

Criterios de validación

- Un usuario válido puede autenticarse y recibir un token.
- Una solicitud sin token no puede acceder a recursos protegidos.
- Un token inválido o expirado es rechazado.
- El frontend redirige o informa correctamente cuando la sesión no es válida.
- No se exponen credenciales en respuestas, logs ni código fuente.

Riesgos y consideraciones

- Configuración incorrecta de expiración o firma del token.
- Manejo inseguro del token en el navegador.
- Diferencias entre la configuración de desarrollo y producción.

3. Alta y Baja de Equipos

Semana programada: S3 — 12 al 18 de octubre | Responsables: Ángel y Miguel

Objetivo

Construir el módulo para registrar y retirar equipos del sistema, manteniendo la integridad de la información y dejando preparado el historial necesario para auditoría.

Descripción de la ejecución

El módulo permitirá administrar el catálogo de equipos. Se debe definir qué campos son obligatorios, qué datos identifican de manera única a cada equipo y qué significa exactamente dar de baja. Cuando el negocio lo permita, se priorizará una baja lógica para conservar trazabilidad.

Procedimiento de trabajo

26. Definir los campos del equipo, incluyendo identificador, nombre o descripción, estado y datos adicionales requeridos.
27. Implementar validaciones de negocio para evitar duplicados o registros incompletos.
28. Crear endpoints de alta y baja en la API .NET.
29. Aplicar validaciones también en Angular para mejorar la experiencia, sin sustituir las validaciones del backend.
30. Construir formularios con mensajes de error claros.
31. Implementar confirmación antes de ejecutar una baja.
32. Definir si la baja será lógica o física; preferentemente conservar el registro si se requiere trazabilidad.
33. Actualizar el listado después de una operación exitosa sin requerir recarga innecesaria.
34. Preparar eventos o registros que posteriormente puedan ser utilizados por el módulo de auditoría.
35. Probar escenarios de alta válida, duplicado, campos vacíos, baja y recuperación de errores.

Entregables

- Modelo de datos de equipos.
- Endpoints de alta y baja.
- Pantallas/formularios Angular.
- Validaciones frontend y backend.
- Mensajes y estados de operación.
- Casos de prueba funcionales.

Criterios de validación

- Se puede registrar un equipo válido.
- No se aceptan identificadores duplicados cuando deban ser únicos.
- La baja requiere confirmación.
- Los registros dados de baja no aparecen como activos.
- Los errores del servidor se muestran de forma comprensible.

Riesgos y consideraciones

- Definir una baja física podría eliminar información necesaria para auditoría.
- Reglas de negocio ambiguas pueden provocar registros inconsistentes.
- La validación únicamente en frontend permitiría datos inválidos mediante llamadas directas a la API.

4. Búsqueda y Edición

Semana programada: S4 — 19 al 25 de octubre | Responsables: Ángel y Emiliano

Objetivo

Permitir localizar equipos de manera rápida y modificar la información autorizada sin comprometer la integridad de los datos.

Descripción de la ejecución

Esta fase amplía la administración de equipos. La búsqueda debe ser práctica aun cuando el número de registros aumente, mientras que la edición debe controlar qué campos pueden modificarse y validar los cambios antes de persistirlos.

Procedimiento de trabajo

36. Definir los criterios de búsqueda que tendrá el sistema.
37. Implementar filtros en la API evitando traer innecesariamente todos los registros al navegador.
38. Agregar paginación si el volumen de información lo requiere.
39. Crear la interfaz de resultados en Angular.
40. Agregar acciones de edición asociadas a cada registro.
41. Cargar los datos actuales en un formulario reutilizable.
42. Validar campos obligatorios, formatos y restricciones de negocio.
43. Enviar únicamente los cambios permitidos al backend.
44. Actualizar la interfaz al terminar una edición exitosa.
45. Registrar errores de concurrencia o conflictos de actualización cuando correspondan.

Entregables

- Endpoint de búsqueda y filtros.
- Interfaz de listado.
- Formulario de edición.
- Validaciones.
- Manejo de errores y estados de carga.
- Pruebas de búsqueda y actualización.

Criterios de validación

- Los criterios de búsqueda producen resultados correctos.
- La interfaz diferencia carga, resultados y ausencia de resultados.
- Una edición válida persiste correctamente.
- No se pueden modificar campos restringidos desde el cliente.
- La API valida nuevamente los datos recibidos.

Riesgos y consideraciones

- Traer demasiados registros puede afectar rendimiento.
- Permitir edición masiva sin controles puede provocar cambios accidentales.
- Dos usuarios podrían intentar modificar el mismo registro simultáneamente.

5. Dashboard en Tiempo Real

Semana programada: S5 — 26 de octubre al 1 de noviembre | Responsables: Ángel y Emiliano

Objetivo

Construir un dashboard que presente indicadores y estados relevantes del sistema y que pueda actualizar información sin depender de recargas manuales.

Descripción de la ejecución

El dashboard concentrará información operacional. Para la actualización en tiempo real se definirá un canal de comunicación adecuado a la arquitectura .NET, por ejemplo un mecanismo basado en SignalR cuando los requerimientos del proyecto lo justifiquen.

Procedimiento de trabajo

46. Definir los indicadores que se mostrarán y su fuente de datos.
47. Separar métricas de resumen de información detallada.
48. Diseñar los endpoints o consultas de agregación necesarios.
49. Definir qué eventos deben provocar una actualización del dashboard.
50. Implementar el canal de comunicación en tiempo real cuando corresponda.
51. Crear servicios Angular para consumir los datos.
52. Construir tarjetas, tablas o gráficas según las métricas definidas.
53. Agregar estados de carga, conexión, reconexión y error.
54. Evitar actualizaciones innecesarias mediante una estrategia de eventos o actualización controlada.
55. Validar que los valores mostrados coincidan con los datos persistidos.

Entregables

- Dashboard funcional.
- Consultas/endpoints de indicadores.
- Mecanismo de actualización en tiempo real.
- Componentes visuales Angular.
- Manejo de conexión y errores.
- Pruebas de consistencia de datos.

Criterios de validación

- Los indicadores corresponden a datos reales del sistema.
- Los cambios relevantes se reflejan sin recarga manual cuando se haya definido tiempo real.
- La interfaz indica cuando existe un problema de conexión.
- El dashboard mantiene un rendimiento aceptable con el volumen de datos de prueba.

Riesgos y consideraciones

- Enviar demasiados eventos puede saturar clientes o servidor.
- Consultas de agregación pesadas pueden degradar SQL Server.
- Un dashboard visualmente correcto puede mostrar datos incorrectos si no se validan las fuentes.

6. Roles y Logs

Semana programada: S6 — 2 al 8 de noviembre | Responsables: Emiliano y Miguel

Objetivo

Incorporar control de acceso por roles y un registro de actividades relevantes para conocer quién realizó una acción, cuándo y sobre qué recurso.

Descripción de la ejecución

La autorización debe diferenciar autenticación de permisos. Un usuario autenticado no necesariamente tendrá acceso a todas las operaciones. La auditoría permitirá conservar evidencia técnica de acciones relevantes y facilitar análisis de incidencias.

Procedimiento de trabajo

56. Definir los roles y permisos necesarios para el sistema.
57. Relacionar usuarios con roles mediante el modelo de datos.
58. Incluir claims o información de rol en el mecanismo de autenticación de acuerdo con la arquitectura.
59. Aplicar autorización en endpoints y servicios del backend.
60. Ocultar o deshabilitar en Angular las acciones que el usuario no tenga permitido realizar.
61. Definir las operaciones que deben registrarse en auditoría.
62. Crear la estructura de logs con usuario, acción, fecha/hora, recurso y resultado, evitando información sensible.
63. Registrar eventos relevantes desde el backend para que el cliente no pueda falsear la auditoría.
64. Crear consultas o una vista de auditoría para usuarios autorizados.
65. Probar accesos permitidos y denegados con diferentes roles.

Entregables

- Matriz de roles y permisos.
- Autorización en API.
- Controles de interfaz en Angular.
- Tabla o mecanismo de auditoría.
- Vista/consulta de logs.
- Casos de prueba de permisos.

Criterios de validación

- Cada endpoint protegido valida el permiso correspondiente.
- Un usuario sin permiso recibe una respuesta de autorización adecuada.
- Las acciones auditables generan registros.
- Los logs no almacenan contraseñas, tokens completos ni secretos.
- La auditoría permite reconstruir las operaciones relevantes.

Riesgos y consideraciones

- Confiar únicamente en controles visuales del frontend.
- Registrar información sensible en logs.
- Definir permisos demasiado amplios o ambiguos.

7. Refinamiento DTUI

Semana programada: S7 — 9 al 15 de noviembre | Responsables: Ángel

Objetivo

Refinar la interfaz de usuario y la experiencia de interacción, asegurando consistencia visual, claridad de estados y comportamiento adecuado en diferentes resoluciones.

Descripción de la ejecución

El refinamiento se realizará después de que los módulos principales sean funcionales. El objetivo es mejorar la interfaz sin modificar innecesariamente la lógica de negocio.

Procedimiento de trabajo

66. Revisar navegación, jerarquía visual y consistencia entre pantallas.
67. Homogeneizar botones, formularios, tablas, tarjetas, mensajes y estados.
68. Revisar estados de carga, vacío, error, éxito y confirmación.
69. Mejorar la distribución de los componentes para escritorio y resoluciones menores.
70. Revisar accesibilidad básica: contraste, etiquetas, foco, textos y navegación mediante teclado.
71. Reducir elementos visuales innecesarios que distraigan del flujo principal.
72. Validar que los componentes reutilizables tengan un comportamiento consistente.
73. Revisar textos, mensajes de validación y nomenclatura.
74. Realizar una revisión visual cruzada con los módulos ya desarrollados.
75. Aplicar correcciones y documentar cambios que afecten componentes compartidos.

Entregables

- Interfaz refinada.
- Componentes visuales consistentes.
- Correcciones responsive.
- Mensajes y estados de interacción revisados.
- Lista de ajustes aplicados.

Criterios de validación

- Las pantallas mantienen una apariencia coherente.
- Los estados de carga/error/éxito son claros.
- La interfaz es usable en las resoluciones objetivo.
- No se rompen funcionalidades existentes después del refinamiento.

Riesgos y consideraciones

- Cambios visuales pueden afectar componentes reutilizados.
- Refactorizar demasiado cerca de QA puede introducir regresiones.
- Priorizar estética sobre claridad funcional.

8. Pruebas Generales y QA

Semana programada: S8 — 16 al 22 de noviembre | Responsables: Miguel (Líder) y equipo

Objetivo

Verificar de forma integral que el sistema cumpla con los requisitos funcionales, técnicos y de seguridad definidos antes de producción.

Descripción de la ejecución

QA debe evaluar el sistema como un producto integrado y no como módulos aislados. Se revisarán flujos completos, permisos, persistencia, errores, rendimiento básico, compatibilidad y regresiones.

Procedimiento de trabajo

76. Construir una matriz de casos de prueba a partir de los requisitos y entregables.
77. Probar inicio y cierre de sesión, expiración de token y accesos protegidos.
78. Probar alta, baja, búsqueda y edición de equipos.
79. Verificar dashboard y actualización de datos.
80. Validar roles, permisos y registros de auditoría.
81. Realizar pruebas de datos inválidos, campos vacíos, duplicados y operaciones no autorizadas.
82. Ejecutar pruebas de integración entre Angular, .NET y SQL Server.
83. Realizar pruebas de regresión después de cada corrección importante.
84. Registrar defectos con pasos para reproducirlos, evidencia, severidad y estado.
85. Repetir las pruebas después de las correcciones y preparar el reporte de aceptación.

Entregables

- Plan/matriz de pruebas.
- Evidencias de ejecución.
- Registro de incidencias.
- Reporte de resultados de QA.
- Lista de pendientes aceptados o corregidos.
- Criterios de liberación para producción.

Criterios de validación

- Los casos críticos pasan satisfactoriamente.
- No existen defectos bloqueantes abiertos para producción.
- Los permisos funcionan conforme a la matriz definida.
- Los flujos principales funcionan de principio a fin.
- Los defectos corregidos cuentan con una prueba de regresión.

Riesgos y consideraciones

- Probar únicamente casos exitosos.
- No repetir pruebas después de correcciones.
- No diferenciar defectos críticos de mejoras menores.

10. Integración entre las fases

Las fases no deben considerarse completamente independientes. El resultado de una actividad sirve como entrada para la siguiente. La siguiente secuencia resume las dependencias principales.

Origen	Entrega que habilita	Siguiente fase
S1 — Arquitectura y BD	Modelo de datos, API base y conexión a SQL Server	S2 — JWT
S2 — JWT	Usuarios autenticados y endpoints protegidos	S3 — Alta/Baja
S3 — Alta/Baja	Gestión inicial de equipos	S4 — Búsqueda/Edición
S4 — Búsqueda/Edición	Administración completa del catálogo	S5 — Dashboard
S5 — Dashboard	Indicadores y actualización de información	S6 — Roles/Logs
S6 — Roles/Logs	Control de acceso y trazabilidad	S7 — DTUI
S7 — DTUI	Interfaz refinada	S8 — QA
S8 — QA	Versión candidata validada	S9 — Producción

10.1 Flujo de integración por módulo

- Angular consume los endpoints publicados por .NET y no accede directamente a SQL Server.
- .NET concentra autenticación, autorización, validaciones de negocio y acceso a datos.
- SQL Server conserva la información persistente y aplica restricciones de integridad.
- Los eventos o actualizaciones en tiempo real deben originarse en el backend y reflejarse en Angular.
- La auditoría se genera preferentemente en el backend para mantener la confiabilidad del registro.

11. Estrategia de pruebas

Tipo	Qué se verifica	Momento
Unitarias	Funciones, servicios y lógica aislada	Durante desarrollo
Integración	Angular ↔ API ↔ SQL Server	S3–S8
Funcionales	Flujos completos del usuario	S4–S8
Seguridad	JWT, roles y accesos no autorizados	S2 y S6–S8
Regresión	Que una corrección no rompa funciones existentes	S4–S9
Smoke test	Funciones críticas después del despliegue	S9

12. Gestión de incidencias

Toda incidencia encontrada durante desarrollo o QA deberá registrarse con suficiente información para reproducirla y darle seguimiento.

Campo	Contenido esperado
ID	Identificador único de la incidencia.
Título	Descripción breve y concreta.
Módulo	Funcionalidad afectada.
Pasos	Procedimiento para reproducir el problema.
Resultado esperado	Comportamiento correcto.
Resultado actual	Comportamiento observado.
Severidad	Bloqueante, alta, media o baja, según los criterios acordados.
Evidencia	Captura, log o información técnica relevante.
Responsable	Persona encargada de corregir.
Estado	Abierta, en desarrollo, corregida, en validación o cerrada.

13. Criterios generales de aceptación

- El sistema permite autenticarse y controlar el acceso de acuerdo con los roles definidos.
- Las operaciones principales sobre equipos son funcionales y validan los datos.
- La información se persiste correctamente en SQL Server.
- El dashboard presenta información coherente con los datos del sistema.
- Las acciones relevantes quedan registradas en auditoría.
- Los errores se manejan de manera controlada y no exponen información sensible.
- La interfaz es consistente y usable en las resoluciones objetivo.
- Los casos críticos definidos por QA se encuentran aprobados.
- La versión de producción corresponde a una versión validada y existe un procedimiento de recuperación.

14. Checklist

☐ Código de frontend y backend correspondiente a la versión candidata.

☐ Scripts/migraciones de SQL Server probados.

☐ Variables de entorno configuradas para producción.

☐ Secretos fuera del repositorio.

☐ JWT configurado con parámetros de producción.

☐ Usuarios y roles iniciales revisados.

☐ Pruebas críticas de QA aprobadas.

☐ Respaldo de la base de datos realizado.

☐ Configuración HTTPS revisada.

15. Conclusión

El cronograma que se plantea organiza el desarrollo del sistema “IO” en nueve fases que avanzan desde la definición de arquitectura y datos hasta la publicación en producción. La estrategia propuesta busca que cada semana genere un entregable verificable y que las actividades posteriores se apoyen en componentes previamente validados. La coordinación entre Angular, .NET y SQL Server, junto con la incorporación progresiva de autenticación, administración, tiempo real, auditoría, refinamiento visual, QA y despliegue, permite conducir el proyecto de manera ordenada y reducir el riesgo de integrar componentes incompletos al final.

Anexo A. Resumen ejecutivo por semana

Semana	Actividad principal	Resultado esperado
S1	Arquitectura y BD	Base técnica y datos listos
S2	JWT	Acceso seguro a la aplicación
S3	Alta/Baja	Administración inicial de equipos
S4	Búsqueda/Edición	Gestión completa de información
S5	Dashboard	Visualización y actualización en tiempo real
S6	Roles/Logs	Permisos y trazabilidad
S7	DTUI	Interfaz refinada y consistente
S8	QA	Versión validada
S9	Producción	Sistema desplegado y verificado

Anexo B. Responsabilidades por fase

Responsable(s)	Participación principal
Ángel	Arquitectura/BD, autenticación, administración de equipos, búsqueda/edición, dashboard y refinamiento DTUI según fase.
Emiliano	Arquitectura/BD, autenticación, búsqueda/edición, dashboard, roles/logs y soporte de integración según fase.
Miguel	Arquitectura/BD, alta/baja, roles/logs, liderazgo de QA y validación.
Equipo completo	Integración, QA, correcciones finales y despliegue a producción.